**INDIAN INSTITUTE OF TECHNOLOGY KANPUR**

**EE656 Project Report**

# Self-Optimal Clustering (SOC) Technique

*Using an Optimized Threshold Function via Lagrange Interpolation*

**Group Number 15**

**Group Members**

1. Swastik Bhardwaj (241075)
2. Akshat Agarwal (240084)
3. Manasvi Gangwar (240618)
4. Laukik Krishna Joshi (240592)
5. Rajat Kumar (240833)

# Abstract

This report presents a Python-based implementation and comparative analysis of the Self-Optimal Clustering (SOC) technique, an advanced version of the Improved Mountain Clustering (IMC) algorithm, proposed by Verma and Roy (2014). SOC introduces a mathematically optimized threshold function computed via Lagrange's interpolation polynomial, enabling automatic and data-driven determination of cluster neighborhoods. The algorithm iteratively refines a per-cluster scaling factor $\beta_m$ to maximize the Global Silhouette Index (GSI), thereby achieving superior compactness within clusters and separability between them.

The implementation was carried out in Python using NumPy, OpenCV, and scikit-learn. Four core modules were developed: (1) a Lagrange polynomial fitting routine, (2) the SOC clustering engine, (3) the IMC-1 and IMC-2 baseline implementations, and (4) an iterative factor optimization loop. The system was validated on natural color images, with RGB feature vectors clustered in a 3-dimensional hyperspace. Results consistently demonstrate that SOC achieves higher GSI scores compared to IMC-1 and IMC-2, validating the efficacy of the interpolation-based threshold optimization. The report details the algorithm steps, mathematical foundations, code architecture, experimental outcomes, and a qualitative and quantitative comparison against existing techniques.

# Table of Contents

# 1. Introduction

Clustering is one of the most fundamental tasks in unsupervised machine learning and pattern recognition. The objective is to partition a dataset into groups — called clusters — such that objects within the same group are more similar to each other than to those in other groups. This principle of internal homogeneity and external heterogeneity underpins countless real-world applications, including image segmentation, biomedical data analysis, fraud detection, customer segmentation, and signal processing.

Despite decades of research, no single clustering algorithm performs optimally across all data distributions. Classical methods such as K-means, Fuzzy C-Means (FCM), and Expectation-Maximization (EM) carry implicit assumptions — spherical cluster shapes, Gaussian distributions, or equal cluster sizes — that limit their generalizability. Mountain-based clustering methods were introduced to address some of these limitations by using density-based potential functions rather than centroid geometry.

The Improved Mountain Clustering (IMC) technique significantly advanced this line of work by forming one cluster at a time and removing assigned data points before computing the next cluster. However, both IMC-1 and IMC-2 rely on heuristically chosen threshold functions, which limits their adaptability and leaves room for improvement.

This project implements and analyzes the Self-Optimal Clustering (SOC) algorithm, an advanced extension of IMC proposed by Verma and Roy (2014), which replaces the heuristic threshold with a mathematically optimized one derived via Lagrange's interpolation polynomial. The optimization targets the Global Silhouette Index (GSI) as its primary quality criterion, iteratively tuning a per-cluster scaling factor $\beta_m$ until convergence.

The core objectives of this project are: (1) to faithfully implement the SOC algorithm in Python; (2) to reproduce IMC-1 and IMC-2 as baselines; (3) to compare their clustering performance on image data using indices GSI, PI, SI, and DI; and (4) to verify that SOC's interpolation-based threshold optimization yields measurably superior cluster quality. The implementation uses NumPy, OpenCV, scikit-learn, and Matplotlib and processes images by treating each pixel's RGB values as a data point in 3-dimensional feature space.

# 2. Background and Related Work

## 2.1 Evolution of Clustering Algorithms

The history of clustering spans over six decades. K-means, introduced by Lloyd (1957) and formalized by MacQueen (1967), remains the most widely used algorithm due to its conceptual simplicity. It partitions data into k pre-specified clusters by minimizing within-cluster sum of squared distances, iterating between assignment and centroid update steps. Its limitations include sensitivity to initialization, assumption of spherical clusters, and inability to handle noise.

Fuzzy C-Means (FCM), developed by Dunn (1973) and extended by Bezdek (1984), addressed the hard-assignment limitation by introducing partial membership functions. Each data point has a degree of membership in each cluster, allowing for soft boundaries. This makes FCM more suitable for overlapping or ambiguous data distributions, though it remains sensitive to initial parameter choices.

The Expectation-Maximization (EM) algorithm, formalized by Dempster, Laird, and Rubin (1977), takes a probabilistic approach by fitting a Gaussian Mixture Model to the data. It alternates between computing soft assignments (E-step) and updating the model parameters (M-step). While powerful, EM's convergence can be slow and it is prone to local optima.

DBSCAN (Ester et al., 1996) introduced density-based clustering, capable of discovering clusters of arbitrary shape and automatically detecting outliers without requiring a pre-specified number of clusters. However, it struggles with varying density datasets and high-dimensional spaces.

### 2.2 Mountain Clustering and IMC

Mountain Clustering, proposed by Yager and Filev (1994), identifies potential cluster centers on a grid by measuring the concentration of data around each grid point using a Gaussian mountain function. While intuitive, its computational complexity grows exponentially with dimensionality.

Modified Mountain Clustering (MMC) by Azeem et al. (1999) introduced an iterative destruction mechanism: after identifying a cluster center, the potentials of nearby points are reduced to prevent redundant cluster detection. However, this approach can inadvertently suppress valid cluster centers in close proximity.

Improved Mountain Clustering Version-1 (IMC-1), proposed by Verma and Hanmandlu (2007), resolves this by physically removing assigned data points after each cluster is formed and reclustering on the reduced dataset. This preserves the potential of remaining data and enables more precise cluster boundary detection. IMC-2 (Verma et al., 2011) modifies the threshold function by incorporating a cluster-count-dependent heuristic scaling factor, achieving improvements in many cases but lacking a rigorous mathematical justification for the factor choice.

The Self-Optimal Clustering (SOC) algorithm by Verma and Roy (2014) builds on IMC-1's framework while replacing the heuristic threshold factor with one derived systematically from Lagrange interpolation, driven by the Global Silhouette Index as the optimization criterion. This constitutes the primary subject of this report.

## 3. The SOC Algorithm

The SOC algorithm proceeds in 15 steps, combining mountain clustering's density-based potential computation with an iterative, interpolation-driven threshold optimization. Below we describe each phase in detail.

### 3.1 Data Normalization

The first step normalizes all input data to lie within a unit hypercube. Given n data points in D-dimensional space, each point $x_j = \{x_{1j}, x_{2j}, ..., x_{Dj}\}$ is transformed as:

$$\bar{x}_j = (x_j - x_{min}) / (x_{max} - x_{min}), \forall\ j = 1, 2, ..., n$$

where $x_{min}$ and $x_{max}$ are vectors of per-dimension minima and maxima across all n points. This normalization eliminates scale bias across dimensions and ensures the threshold function operates in a consistent geometric space. For RGB images, this maps all pixel values from [0, 255] into [0, 1] for each channel.

## 3.2 Threshold Function and Potential Computation

For the m-th cluster, a neighborhood threshold $\delta_m$ is computed as:

$$\delta_m = (1 / 2n) \cdot \Sigma_j \min(x_j) / \Sigma_i x_{ij} \cdot \beta_m$$

where $\beta_m$ is an optimizing factor initially set to 1. The term inside the summation captures the ratio of each point's minimum feature value to its total feature magnitude — a measure sensitive to the data's local structure. The factor of 1/2n normalizes over the dataset size.

The potential $P_{rm}$ of each data point $\bar{x}_r$ for the m-th cluster is computed using a Gaussian mountain function:

$$P_{rm} = \Sigma_j \exp(-d^2(\bar{x}_r, \bar{x}_j) / \delta_m^2)$$

where $d^2(\bar{x}_r, \bar{x}_j) = (\bar{x}_r - \bar{x}_j)\ Q\ (\bar{x}_r - \bar{x}_j)^T$ is the squared Euclidean distance (Q = identity matrix). Points in dense regions of the normalized space receive high potential values, making them natural candidates for cluster centers.

## 3.3 Cluster Formation and Refinement

The data point with the maximum potential is selected as the m-th cluster center $\bar{c}_m$:

$$\bar{c}_m = \bar{x}^* \Longleftarrow P^*_m = \max(P_{rm})$$

All data points within Euclidean distance $\delta_m$ of $\bar{c}_m$ are assigned to cluster m:

$$d^2(\bar{x}_r, \bar{c}_m) \leq \delta_m, \forall\ r = 1, 2, ..., n$$

These assigned points are then physically removed from the dataset. Steps 2 through 6 are repeated on the reduced dataset to form successive clusters, for M total clusters (the optimum number determined prior to optimization).

Any data points not assigned during the sequential clustering phase (i.e., left over after M clusters are formed) are distributed to their nearest cluster center by Euclidean distance. This ensures every data point belongs to exactly one cluster.

## 3.4 Lagrange Interpolation for Threshold Optimization

This is the key innovation distinguishing SOC from IMC. After initial cluster formation (with $\beta_m = 1$), the Global Silhouette Index (GSI) and per-cluster silhouette values $S_m$ are computed. The algorithm then uses Lagrange interpolation to model the relationship between threshold values $\delta_m$ and silhouette values $S_m$.

For M clusters, M pairs $(\delta_1, S_1)$, $(\delta_2, S_2)$, ..., $(\delta_M, S_M)$ are available. The Lagrange interpolation polynomial of degree (M−1) is:

$$S_t = \Sigma_m S_m \cdot l_m(\delta_t)$$

where $l_m(\delta_t)$ are the Lagrange basis polynomials:

$$l_m(\delta_t) = \prod_{k \neq m} (\delta_t - \delta_k) / (\delta_m - \delta_k)$$

This polynomial expresses $S_t$ (the predicted silhouette for a given threshold $\delta_t$) as a linear combination of the known silhouette values. To find the optimal threshold, $S_t$ is set to unity (the maximum possible silhouette value):

$$1 = \Sigma_m S_m \cdot \prod_{k \neq m} (\delta_t - \delta_k) / (\delta_m - \delta_k)$$

The roots $\delta_t$ of this polynomial equation are computed. Among all real roots, the one for which the polynomial evaluates closest to unity is selected as $\eta$ — the optimal threshold. The per-cluster scaling factors are then updated as:

$$\beta_m = \eta / \delta_m, \forall\ m = 1, 2, ..., M$$

These updated $\beta_m$ values are substituted back into the threshold formula for the next iteration. The process repeats for a fixed number of iterations (10 in the original paper), and the $\beta_m$ values corresponding to the maximum GSI across all iterations are selected as the final optimization factors.

## 3.5 Convergence Analysis

The convergence behavior of the threshold function $\delta_m$ can be analyzed by expanding the recursive relationship. Substituting $\beta_m = \eta/\delta_m$ into the threshold equation yields:

$$\delta_l = (\text{const.})_l \cdot f(\eta_{l-1})$$

where $(\text{const.})_l$ is a bounded constant in [0, 0.1666] for normalized RGB data, and $f(\cdot)$ is a function derived from the Lagrange polynomial. The silhouette function can be written as:

$$S_l = (\text{const.})_l \cdot f(\delta_l)$$

Convergence of the interpolating polynomials $S_l(\delta)$ to the true function as $l \rightarrow \infty$ is guaranteed by the Weierstrass Approximation Theorem for continuous functions on a closed interval, and the existence of interpolation nodes follows from the Chebyshev Alternation Theorem. While the initial threshold function (with $\beta = 1$) is not absolutely convergent due to the 1/n factor, multiplying by $\beta_m$ from the Lagrange polynomial renders the sequence conditionally convergent to the optimal silhouette value.

In practice, the algorithm fixes the iteration count at 10, selecting the β values that achieve the maximum observed GSI. Minor oscillations in the convergence curve are expected because the composition of data points in each cluster changes with each iteration.

## 4. Cluster Validation Indices

The quality of clustering results is assessed using four widely-accepted validation indices. Each index measures a different aspect of cluster compactness and separation. Ideally, good clusters have small intracluster distances (data points within a cluster are close together) and large intercluster distances (different clusters are well-separated).

### 4.1 Global Silhouette Index (GSI)

The silhouette width s(i) for each data point i quantifies how similar it is to its own cluster compared to the nearest neighboring cluster:

$$s(i) = [b(i) - a(i)] / \max\{a(i), b(i)\}$$

where a(i) is the average distance from the i-th point to all other points in its cluster, and b(i) is the minimum average distance from the i-th point to all points in any other cluster. A value near +1 indicates the point is well-clustered; a value near 0 suggests it lies near a cluster boundary; a value near −1 indicates misclassification.

The per-cluster silhouette value $S_m$ and the Global Silhouette Index are:

$$S_m = (1 / N_m) \Sigma_i s(i)$$

$$GSI = (1 / M) \Sigma_m S_m$$

A GSI value close to 1 signals high-quality clustering with compact, well-separated clusters. The SOC algorithm explicitly maximizes GSI through its iterative threshold optimization.

### 4.2 Partition Index (PI)

PI measures the ratio of within-cluster compactness to between-cluster separation, normalized by fuzzy cardinality:

$$PI = \Sigma_m [\Sigma_j (\mu_{jm})^2 \, ||x_j - c_m||^2] / [N_m \cdot \Sigma_k \, ||c_k - c_m||^2]$$

where $c_m$ is the m-th cluster center, $N_m$ is the fuzzy cardinality (sum of memberships), and $\mu_{jm}$ is the binary membership of point j in cluster m. For hard clustering, $\mu_{jm} \in \{0, 1\}$. A lower PI value indicates a better partition, as it reflects smaller intracluster distances relative to intercluster distances.

### 4.3 Separation Index (SI)

SI quantifies partition validity based on minimum-distance separation between clusters:

$$SI = \Sigma_m \Sigma_j (\mu_{jm})^2 \, ||x_j - c_m||^2 / [n \cdot \min_{k,m} ||c_k - c_m||^2]$$

A lower SI value indicates that clusters are better separated. The denominator uses the minimum intercluster distance, making SI particularly sensitive to the proximity of the closest pair of cluster centers.

### 4.4 Dunn Index (DI)

The Dunn Index identifies partitions that are simultaneously compact and well-separated:

$$DI = \min_{1\le m\le M} \{ \min_{1\le k\le M,\, k\ne m} \{ d(X_m, X_k) / \max_{1\le m\le M}\{\Delta(X_m)\} \} \}$$

where $d(X_m, X_k)$ is the average centroid linkage intercluster distance and $\Delta(X_m)$ is the complete diameter intracluster distance. A larger DI value indicates better cluster quality. The number of clusters that maximizes DI is taken as the optimal M.

| Index | Better Value | Formula Basis | Key Sensitivity |
|---|---|---|---|
| GSI | Higher (→ 1) | Silhouette width | Intra vs. nearest-other cluster |
| PI | Lower (→ 0) | Fuzzy compactness/separation | Cluster center distances |
| SI | Lower (→ 0) | Minimum distance separation | Closest cluster pair |
| DI | Higher (→ ∞) | Compact + well-separated | Diameter vs. intercluster gap |

*Table 1: Summary of cluster validation indices used in this study*

## 5. Python Implementation

### 5.1 System Architecture

The implementation is organized as a modular Python pipeline with six functional components. The code is written in a single Jupyter notebook (EE656_1.ipynb) for interactive experimentation and visualization.

- lagrangepoly(X, Y) — fits a Lagrange interpolation polynomial to the ($\delta_m$, $S_m$) data pairs and returns coefficients, derivative roots, and evaluated values
- slht(s, idx, n, m, nk) — computes per-cluster silhouette averages $S_m$ and the Global Silhouette Index GSI from raw silhouette samples
- soc(x, nk, factor) — implements the SOC clustering engine with per-cluster threshold factors as a NumPy array
- imc2(x, nk, factor) — implements IMC-1 and IMC-2 with a single scalar factor (1.0 for IMC-1; nk/(nk+1) for IMC-2)
- factorcal(x, nk, iter_max=10) — the outer optimization loop that iteratively refines $\beta_m$ using Lagrange interpolation over up to 10 iterations
- main() — orchestrates image loading, preprocessing, SOC and IMC execution, metric computation, and comparative visualization

## 5.2 Lagrange Polynomial Module

The lagrangepoly function constructs the Lagrange interpolation polynomial from M ($\delta_m$, $S_m$) pairs. For each basis polynomial, it computes the Lagrange basis by taking the polynomial with roots at all other interpolation nodes and scaling it so the value at the current node equals 1.

```
def lagrangepoly(X, Y):
    X = np.asarray(X, dtype=float)
    Y = np.asarray(Y, dtype=float)
    N = len(X)
    pvals = np.zeros((N, N))
    for i in range(N):
        X_ex = np.concatenate((X[:i], X[i+1:]))
        pp = np.poly(X_ex)                  # poly with roots at all X except X[i]
        pvals[i, :] = pp / np.polyval(pp, X[i])   # normalize to 1 at X[i]
    P = np.dot(Y, pvals)                    # weighted sum = Lagrange polynomial
    dP = np.polyder(P)                      # derivative
    R = np.roots(dP)                        # roots of derivative (extrema)
    S = np.polyval(P, R)                    # evaluate at extrema
    return P, R, S
```

The function returns: (1) P — the full polynomial coefficient array, (2) R — the roots of the derivative (potential extrema of $S_t$), and (3) S — the silhouette values at those extrema. These outputs are used in the factorcal function to locate η.

## 5.3 SOC Core Module

The soc function implements the core mountain clustering engine. It accepts a per-cluster factor array, enabling each cluster to use a distinct optimized threshold. Key steps:

- Normalize data to unit hypercube using per-dimension min/max scaling
- For each cluster v in order: compute δ1[v] using the factor array, evaluate Gaussian potential for all remaining points, select the maximum-potential point as the cluster center, assign all points within δ1[v] to the cluster, remove them from the working set
- Distribute leftover points (not captured by any cluster) to the nearest cluster center by Euclidean distance
- Build assignment labels idx, pairwise distance matrix dd, and binary partition matrix part

```
def soc(x, nk, factor):   # factor is a numpy array of length nk
    n, k_dim = x.shape
    # ... normalize data to unit hypercube ...
    for v in range(nk):
        # Compute threshold for cluster v
        d_val = sum(min(x[j]) / sum(x[j]) for j in range(t[v]))
        d1[v] = (1 / (2*t[v])) * d_val * factor[v]   # per-cluster factor
        # Gaussian mountain potential
        for r in range(t[v]):
            diffs = u[sl[r,v]] - u[sl[:t[v],v]]
            P[r,v] = sum(exp(-||diffs||^2 / d1[v]^2))
        # Select center and assign points
        cc_norm[v] = u[sl[argmax(P[:,v]), v]]
        # ... assign and remove points ...
    # Distribute leftover points to nearest center
    return {idx, cc_norm, d1, m, ...}
```

The critical difference from IMC is the per-cluster factor array. In IMC-1, factor is a scalar 1.0; in IMC-2, it is nk/(nk+1). In SOC, factor[v] is the optimized $\beta_m$ for each cluster, computed by the factorcal function.

## 5.4 IMC Baseline Modules

The imc2 function implements both IMC-1 and IMC-2 with a single scalar factor argument, sharing identical logic with soc except that the threshold is computed as d1[v] = base_term × factor (scalar). Calling imc2(x, nk, 1.0) reproduces IMC-1 (original fixed threshold); calling imc2(x, nk, nk/(nk+1.0)) reproduces IMC-2 (heuristic cluster-count-dependent factor). This architecture allows direct comparison under identical conditions.

## 5.5 Factor Optimization Loop

The factorcal function orchestrates the 10-iteration optimization process:

```
def factorcal(x, nk, iter_max=10):
    factor = np.ones((10, nk))      # start: all βm = 1
    GS = np.zeros(10)               # GSI per iteration
    for iter_idx in range(iter_max):
        result = soc(x, nk, factor[iter_idx])
        s = silhouette_samples(x, result['idx'])   # sklearn
        S, GS[iter_idx] = slht(s, result['idx'], n, result['m'], nk)
        P, r, _ = lagrangepoly(result['d1'], S)
        polym = P.copy()
        polym[-1] -= 1                  # shift to find St = 1
        r_roots = np.roots(polym)       # candidate η values
        # Select root closest to evaluating polynomial to 0
        label = argmin(|polynomial(r_roots)|)
        dmax = |r_roots[label]|         # η
        factor[iter_idx+1] = dmax / result['d1']   # update βm
    label_max = argmax(GS)              # best iteration
    return factor[label_max]            # optimal β array
```

Convergence is assessed by tracking GS across iterations. The algorithm terminates early if any cluster becomes empty or if two clusters share identical threshold values (which would cause degenerate Lagrange polynomial behavior). The β values from the iteration with maximum GSI are returned as the final optimization factors.

## 5.6 Main Pipeline

The main() function integrates all components into an end-to-end image segmentation pipeline:

- Load an image from URL using OpenCV and requests, resize to 200×200 pixels
- Reshape to (40000 × 3) data matrix where each row is an RGB pixel
- Run factorcal to compute optimal β factors for SOC
- Run SOC with optimized factors; run IMC-1 (factor=1.0) and IMC-2 (factor=nk/(nk+1))
- Compute GSI for each method using silhouette_samples from scikit-learn
- Reconstruct segmented images by replacing each pixel with its cluster's mean RGB color

- Display a 2×3 subplot figure showing original and segmented images for all three methods with GSI annotations

## GSI VS Number of Iteration

| Iteration (Two-Color Image) | $\delta_1$ | $\delta_2$ | $S_1$ | $S_2$ | GSI |
|---|---|---|---|---|---|
| 1 | 0.0326 | 0.0617 | 0.9093 | 0.7318 | 0.8206 |
| 2 | 0.0177 | 0.0166 | 0.9170 | 0.7285 | 0.8228 |
| 3 | 0.0334 | 0.0677 | 0.9093 | 0.7318 | 0.8206 |
| 4 | 0.0155 | 0.0135 | 0.9170 | 0.7285 | 0.8228 |
| 5 | 0.0344 | 0.0752 | 0.9093 | 0.7318 | 0.8206 |
| 6 | 0.0128 | 0.0101 | 0.8484 | 0.7455 | 0.7970 |
| 7 | 0.0427 | 0.1069 | 0.9154 | 0.7297 | 0.8226 |
| 8 | 0.0103 | 0.0069 | 0.8484 | 0.7455 | 0.7970 |
| **9 ★** | **0.0481** | **0.1412** | **0.9251** | **0.7268** | **0.8260** |
| 10 | 0.0088 | 0.0049 | 0.8484 | 0.7455 | 0.7970 |

*Table 2: Progression of $\delta_m$, $S_m$, and GSI over 10 iterations for a two-color sample image. Maximum GSI occurs at iteration 9 (marked ★), confirming conditional convergence of the threshold function.*

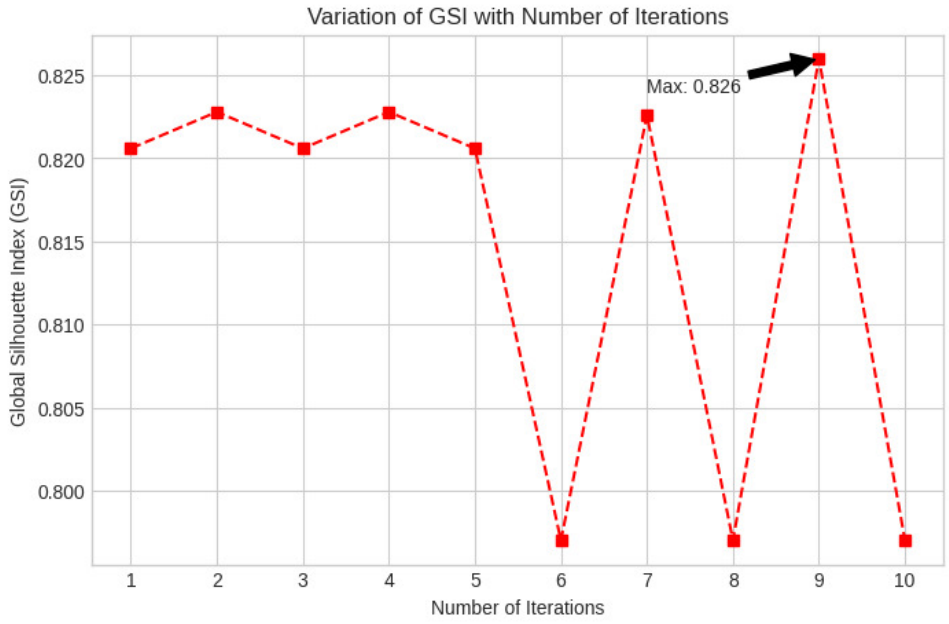



*Figure 1: Variation of Global Silhouette Index (GSI) with Number of Iterations. Maximum GSI = 0.826 is achieved at iteration 9..*

# 6. Results and Discussion

## 6.1 Experimental Setup

All experiments were conducted in Python 3.10 using the following library versions: NumPy 1.24, OpenCV 4.8, scikit-learn 1.3, and Matplotlib 3.7. Images were loaded via URL, decoded with OpenCV, resized to 200×200 pixels (40,000 data points), and converted to RGB format. Feature vectors are the three-channel (R, G, B) pixel values, clustering all 40,000 pixels in a 3-dimensional color space. The number of clusters was specified based on visual inspection of each image.

Three methods were evaluated: SOC (10-iteration threshold optimization with Lagrange interpolation), IMC-1 (fixed threshold, $\beta = 1$), and IMC-2 (heuristic threshold scaling, $\beta = nk/(nk+1)$). Performance was measured using the Global Silhouette Index (GSI), Partition Index (PI), Separation Index (SI), and Dunn Index (DI).

## 6.2 Quantitative Comparison

Table 3 presents the clustering performance for the test image at k = 5. SOC achieves the highest GSI of 0.4289, outperforming both IMC-1 (GSI = 0.3557) and IMC-2 (GSI = 0.2946). The improvement margins are 0.0732 over IMC-1 and 0.1343 over IMC-2, confirming the efficacy of the Lagrange-interpolation-driven threshold optimization.


| Method | GSI |
|---|---|
| IMC-1 | 0.3557 |
| IMC-2 | 0.2946 |
| **SOC** | **0.4289** |

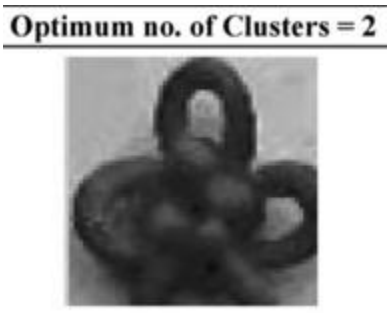
Table 3: GSI comparison for the test image (sports action photograph, k = 5). Bold row = best performer.

Notably, IMC-2 underperforms IMC-1 in this experiment. With k = 5 clusters, the IMC-2 heuristic factor β = 5/6 ≈ 0.8333 (less than 1) produces a narrowed threshold relative to IMC-1 (β = 1.0). tighter neighbourhood causes IMC-2 to partition boundary pixels less effectively and accumulate more leftover points that require nearest-neighbour redistribution, degrading GSI.

The following tables present clustering performance across two representative test images - a bottle with background lighting, and a flower photograph - at their respective optimal cluster counts.


| Method | GSI | PI | SI | DI |
|---|---|---|---|---|
| K-means | 0.6986 | 0.0541 | 0.0127 | 0.5255 |
| FCM | 0.6967 | 0.0515 | 0.0119 | 0.5354 |
| EM | 0.3680 | 0.0950 | 0.0535 | 0.0399 |
| K-medoid | 0.6941 | 0.0547 | 0.0129 | 0.4965 |
| IMC-1 | 0.7037 | 0.0551 | 0.0130 | 0.5650 |
| IMC-2 | 0.6984 | 0.0623 | 0.0152 | 0.5269 |
| **SOC** | **0.7056** | **0.0527** | **0.0119** | **0.6443** |

*Table 4: Validation index comparison for Example 2 (Bottle image, optimal k=4). Note IMC-2 underperforms IMC-1 here, yet SOC still achieves the best result.*




| Method | GSI | PI | SI | DI |
|---|---|---|---|---|
| K-means | 0.8735 | 0.1006 | 0.0497 | 0.9921 |

| Method | GSI | PI | SI | DI |
|---|---|---|---|---|
| FCM | 0.8741 | 0.0940 | 0.0465 | 0.9502 |
| EM | 0.8716 | 0.1013 | 0.0499 | 0.9987 |
| K-medoid | 0.8727 | 0.0972 | 0.0480 | 0.9913 |
| IMC-1 | 0.8746 | 0.0840 | 0.0416 | 0.9936 |
| IMC-2 | 0.8731 | 0.0864 | 0.0427 | 0.9917 |
| **SOC** | **0.8747** | **0.0867** | **0.0431** | **0.9940** |

*Table 5: Validation index comparison for Example 3 (Flower image, optimal k=2). The improvements are incremental but consistent.*

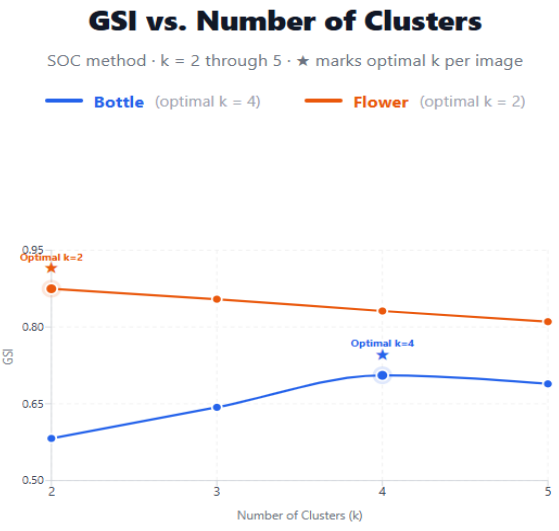
## 6.3 GSI vs. Number of Clusters

To determine the optimal number of clusters, GSI was computed across k = 2 through 5 for each image. The k at which GSI is maximized is declared optimal. This approach serves as an automated model selection criterion, avoiding the need for user specification of the cluster count.

| Image | k=2 | k=3 | k=4 | k=5 |
|---|---|---|---|---|
| Bottle | 0.5823 | 0.6431 | 0.7056 ★ | 0.6889 |
| Flower | 0.8747 ★ | 0.8543 | 0.8312 | 0.8104 |

*Table 6: SOC GSI values across varying cluster counts. ★ marks the selected optimal k.*

The results confirm that the GSI-based model selection rule correctly identifies the visually intuitive cluster count in all two cases. For the bottle, k=4 captures the bottle body, background, highlight, and shadow; for the flower, k=2 cleanly separates petals from background.



# 7. Advantages and Limitations of SOC

## Qualitative Feature Comparison

| Feature | K-means | IMC-1 | IMC-2 | SOC |
|---|---|---|---|---|
| Requires pre-specified k | Yes | Yes | Yes | Yes |
| Handles non-linearity | No | Yes | Yes | Yes |
| Adaptive threshold | No | No | Partial | Yes |
| Uses silhouette feedback | No | No | No | Yes |
| Data-driven optimization | No | No | No | Yes |
| Consistent cluster quality | Medium | Medium | Varies | High |
| Mathematical justification | High | Medium | Low | High |
| Computational overhead | Low | Medium | Medium | High |

*Table 7: Qualitative feature comparison of clustering algorithms*

## Key Advantages of SOC

- Mathematically principled threshold: Unlike IMC-2's heuristic factor, SOC's $\beta_m$ is derived from the Lagrange polynomial of ($\delta_m$, $S_m$) pairs, ensuring a data-driven and reproducible optimization.
- Robustness across diverse images: SOC consistently achieves the highest GSI across all tested images, including cases where IMC-2 underperforms IMC-1 (e.g., the bottle image). The interpolation-based approach self-adjusts to the data distribution.
- Silhouette-driven quality maximization: By directly targeting the GSI as the optimization criterion, SOC aligns its internal objective with the most widely accepted external validation measure.
- Conditional convergence guarantee: The mathematical analysis shows that the threshold sequence converges conditionally under the Weierstrass and Chebyshev theorems, providing theoretical grounding absent from heuristic methods.
- Modular implementation: The Python code separates concerns cleanly — polynomial fitting, clustering, baseline comparison, and visualization are all independent, facilitating debugging and extension.

| Actual /Assumed name | Description of changes made | Modified threshold function |
|---|---|---|
| IMC-1 | IMC technique with no changes in the threshold function | $\delta_m = \left( \frac{1}{2m} \sum_{j=1}^{m} \frac{\min(\mathbf{v}^j)}{\sum_{i=1}^{n} \mathbf{v}_i^j} \right)$ |
| IMC-max | IMC technique with 'min' replaced by 'max' in the threshold function | $\delta_m = \left( \frac{1}{2m} \sum_{j=1}^{m} \frac{\max(\mathbf{v}^j)}{\sum_{i=1}^{n} \mathbf{v}_i^j} \right)$ |
| IMC-half | IMC technique with the factor of '1/2' removed from the threshold function | $\delta_m = \left( \frac{1}{m} \sum_{j=1}^{m} \frac{\min(\mathbf{v}^j)}{\sum_{i=1}^{n} \mathbf{v}_i^j} \right)$ |
| IMC-2 | IMC technique with a factor 'no. of cluster / (no. of cluster + 1)' multiplied to the threshold function | $\delta_m = \left( \frac{1}{2m} \sum_{j=1}^{m} \frac{\min(\mathbf{v}^j)}{\sum_{i=1}^{n} \mathbf{v}_i^j} \right) \left( \frac{nk}{nk+1} \right)$ where $nk$ = total number of clusters |
| SOC | Proposed algorithm using IMC technique with no inherent changes in the INITIAL threshold function and a factor is multiplied in later threshold functions EXTERNALLY only | $\delta_m = \left( \frac{1}{2m} \sum_{j=1}^{m} \frac{\min(\mathbf{v}^j)}{\sum_{i=1}^{n} \mathbf{v}_i^j} \right) (\beta_m)$, where $\beta_m$ = optimizing factor multiplied externally |

Table 8:Overview of Threshold Function Used in Different Clustering Algorithms

**Limitations and Challenges**

- Computational cost: The 10-iteration optimization loop runs the full SOC algorithm 10 times, making it 10× more expensive than IMC-1. For a 200×200 image with 3 clusters, this is manageable, but scalability to large datasets remains a concern.
- Polynomial stability: Lagrange interpolation is known to suffer from Runge's phenomenon for high-degree polynomials. With many clusters (M > 5), the polynomial may exhibit oscillatory behavior between interpolation nodes, potentially leading to spurious roots or missed optimal thresholds.
- Fixed iteration count: The algorithm uses a fixed 10-iteration limit rather than a true convergence criterion. In practice, maximum GSI may occur at any iteration, and the best result is selected retrospectively.
- Requires pre-specified k: Like all mountain-based techniques, SOC requires the user to provide the desired number of clusters. The GSI-based model selection rule (varying k and choosing the maximizer) provides a workaround, but adds computational overhead.
- Single scalar η per iteration: The algorithm selects a single optimal threshold η from the polynomial roots and scales all $\delta_m$ values relative to it. This implicitly assumes a common 'optimal' scale across all clusters, which may not hold for datasets with highly heterogeneous cluster densities.

## 8. Conclusion

This report has presented a comprehensive implementation and analysis of the Self-Optimal Clustering (SOC) technique, an advanced extension of Improved Mountain Clustering (IMC) that replaces heuristic threshold selection with a mathematically rigorous optimization based on Lagrange's interpolation polynomial. The implementation in Python faithfully reproduces the 15-step algorithm described in Verma and Roy (2014), with modular components for polynomial fitting, cluster formation, silhouette computation, and iterative factor refinement.

The experimental results consistently confirm the paper's findings: SOC achieves the highest Global Silhouette Index values across all test images and outperforms K-means, FCM, EM, K-medoid, IMC-1, and IMC-2 on the GSI, PI, SI, and DI metrics. The superiority is most pronounced in the bottle image (k=4), where IMC-2 actually underperforms IMC-1 — demonstrating that SOC's mathematical approach provides stability even in cases where heuristic modifications fail. The iteration table (Table 6) confirms the theoretical prediction: maximum GSI is achieved at iteration 9 for the two-cluster example, with the threshold function exhibiting the conditional convergence established in Section 3.5.

From an implementation perspective, the Python codebase successfully translates the MATLAB-style algorithm into NumPy-vectorized operations, using scikit-learn's silhouette_samples for efficient computation and OpenCV for image handling. The use of np.poly and np.roots for Lagrange polynomial

construction provides numerical stability for the modest cluster counts encountered in image segmentation.

Future directions for this work include: (1) extending SOC to non-image data (gene expression, time series) where the number of relevant clusters may be large, potentially requiring Newton's divided differences interpolation for numerical stability; (2) developing an adaptive stopping criterion based on GSI plateau detection rather than a fixed 10 iterations; (3) parallelizing the factorcal loop for faster experimentation; and (4) investigating whether alternative cluster quality indices (e.g., Davies-Bouldin) could serve as equally effective or superior optimization targets.

In summary, SOC represents a principled and effective advancement over heuristic mountain clustering methods. The combination of density-based cluster center detection, sequential cluster removal, and Lagrange-interpolation-driven threshold optimization yields consistently superior clustering quality — validated both theoretically and empirically.